

1. **Вопрос:** Максимальная пиковая производительность наиболее мощных современных параллельных вычислительных систем измеряется: в единицах EFLOPs, в десятках PFLOPs, в единицах PFLOPs, в сотнях TFLOPs?

Ответ: в десятках PFLOPs (на 2013 год).

2. **Вопрос:** Производительность компьютера, достигнутая при выполнении некоторой программы, выражена в TFLOPs. Это значение говорит о: среднем количестве операций над вещественными данными, представленными в форме с фиксированной запятой, выполненных за секунду в процессе обработки данной программы; общем числе команд, выполненных за время работы программы; средней скорости выполнения данным компьютером арифметических операций над вещественными числами, представленными в форме с плавающей запятой; средней скорости выполнения данным компьютером арифметических операций над вещественными числами, представленными в форме с плавающей запятой, достигнутой при выполнении данной программы; высокой реальной производительности данного компьютера.

Ответ: среднем количестве операций над вещественными данными, представленными в форме с плавающей запятой, выполненных за секунду в процессе обработки данной программы.

3. **Вопрос:** Умножение двух квадратных плотных вещественных матриц компьютер выполнил за 5 сек с производительностью 50 GFLOPs. Какого размера были матрицы: 500*500, 1000*1000, 2000*2000, 5000*5000, 7000*7000, верного ответа нет?

Ответ: 5000*5000, если перемножение происходило по самому простому алгоритму.

4. **Вопрос:** Сколько кризисов software насчитывается за всю историю развития электронных вычислительных систем: 4, 3, 2, 1, ни одного?

Ответ: 3.

5. **Вопрос:** В каком компьютере функциональные устройства сочетали одновременно принципы конвейерной и параллельной обработки: IBM 704, IBM STRETCH, CDC 6600, CDC 7600, ILLIAC IV, ATLAS, верного ответа нет?

Ответ: CDC 7600.

6. **Вопрос:** Отметьте правильные утверждения на тему машинного представления чисел в современных ЭВМ: все существовавшие до сих пор ЭВМ используют в качестве базовой двоичную систему счисления; машинное эpsilon в основном определяется длиной мантиссы в представлении вещественных чисел; мантисса числа в двоичном представлении - та же, что и мантисса его десятичного логарифма; машинные ноль и эpsilon не могут быть получены с помощью фортран-программы, их следует найти в документации к компьютеру; машинное сложение коммутативно; машинное сложение ассоциативно; машинное умножение коммутативно; машинное умножение ассоциативно?

Ответ: Правильные утверждения:

1. Все существовавшие до сих пор ЭВМ используют в качестве базовой двоичную систему счисления - **неверно**, например "Сетунь" (1958) - 3-ичная система.

2. Машинное эpsilon в основном определяется длиной мантиссы в представлении вещественных чисел - **верно**

3. Мантисса числа в двоичном представлении - та же, что и мантисса его десятичного логарифма - **неверно** (если понимать вопрос буквально)

4. Машинные ноль и эpsilon не могут быть получены с помощью фортран-программы, их следует найти в документации к компьютеру - **неверно** (можно написать программу)

5. Машинное сложение коммутативно — **неверно**

6. машинное сложение ассоциативно - **неверно**

7. Машинное умножение коммутативно - **неверно**

8. Машинное умножение ассоциативно - **неверно**

7. **Вопрос:** Кто из перечисленных ниже людей внёс наибольший вклад в развитие параллельной вычислительной техники: Джон Грей; Сеймур Крэй; Стивен Крейн; Кристиан Рэй; Френсис Дрейк?

Ответ: **Сеймур Крэй** (единственный из них, кто занимался суперкомпьютерами: Грей - психолог, Крейн - поэт, Рэй - певец, Дрейк - мореплаватель)

8. **Вопрос:** Действительные числа в машинном представлении: всегда хранятся точно; всегда хранятся с ненулевой ошибкой округления; хранились на всех существовавших вычислительных системах в двоичном представлении; имеют относительную ошибку округления не более машинного нуля; имеют абсолютную ошибку округления не более машинного эпсилон; иногда хранятся точно?

Ответ: Действительные числа:

1. Всегда хранятся точно – **неверно**

2. Всегда хранятся с ненулевой ошибкой округления – **неверно**

3. Хранились на всех существовавших вычислительных системах в двоичном представлении - **неверно** (пример, "Сетунь")

4. Имеют относительную ошибку округления не более машинного нуля – **верно**

5. Имеют абсолютную ошибку округления не более машинного эпсилон - **неверно** (машинный ноль меньше машинного эпсилон)

6. Иногда хранятся точно - **верно**

9. **Вопрос:** Конвейерное ФУ деления состоит из пяти ступеней, срабатывающих за 2, 5, 3, 1 и 1 такт соответственно. Чему равно наименьшее число тактов, за которое можно обработать 40 пар аргументов на данном устройстве: 1, 5, 12, 40, 207, 212, 480, верного ответа нет?

Ответ: **207** (задержка накапливается на второй операции - по 5 тактов на 40 пар дает 200 + первые 2 такта на первую пару + 3+1+1 на последнюю = 207)

10. **Вопрос:** В конвейерном устройстве есть 7 ступеней, срабатывающих за одну единицу времени каждая. За сколько единиц времени это устройство обработает 7 пар аргументов: 1, 3, 7, 8, 13, 14, верного ответа нет?

Ответ: **13** (первая пара - 7 тактов, после 7 такта первой пары последней паре нужно еще 6 тактов чтобы дойти до конца - всего 7+6 = 13)

11. **Вопрос:** Есть два конвейерных ФУ: сложение (4 ступени) и умножение (6 ступеней), все ступени срабатывают за один такт. За сколько тактов будет выполнена векторная операция $A_i = B_i + C_i * s$, $i=1,2,...,60$, с использованием данных устройств в режиме с зацеплением ФУ: 60, 69, 70, 128, 130, 240, 600?

Ответ: n_1 и n_2 — количество ступеней в функциональных устройствах умножения и сложения. N — число элементов в векторе.

При зацеплении по сути образуется единый конвейер длиной $n_1 + n_2$, следовательно $T = n_1 + n_2 + N - 1$, нет умножения на 2, так как в конвейер подаётся сразу N чисел то время как в следующем номере в каждый отдельный конвейер для сложения и умножения подаётся по N чисел. В условиях задачи $T = 69$, в 2 раза меньше, чем без зацепления.

12. **Вопрос:** Есть два конвейерных ФУ: сложение (4 ступени) и умножение (6 ступеней), все ступени срабатывают за один такт. За сколько тактов будет выполнена векторная операция $A_i = B_i + C_i * s$, $i=1,2,...,60$, с использованием данных устройств без зацепления ФУ: 60, 69, 70, 128, 130, 240, 600?

Ответ: Без зацепления имеем два отличных конвейера. Тогда при последовательной работе ФУ потребуется $T = n_1 + n_2 + 2(N-1)$ такта времени. В условиях данной задачи $T = 128$.

13. **Вопрос:** Архитектура компьютеров. Отметьте правильные утверждения: в SMP-компьютерах все процессоры равноправны; архитектуры NUMA и ccNUMA позволяют сохранить единое адресное пространство для параллельной программы; кэш-память явилась причиной возникновения архитектуры NUMA; поиск команд, которые можно выполнять параллельно, в суперскалярных процессорах происходит во время работы программы; параллелизм в классических VLIW-компьютерах выделяется компилятором.

Ответ: В SMP-компьютерах все процессоры равноправны — да, архитектура SMP означает, что есть нескольких однородных процессоров и массив общей памяти (обычно из нескольких независимых блоков). Все процессоры имеют доступ к любой точке памяти с одинаковой скоростью. Процессоры подключены к памяти либо с помощью общей шины (базовые 2-4 процессорные SMP-сервера), либо с помощью crossbar-коммутатора (HP 9000).

Архитектуры NUMA и ccNUMA позволяют сохранить единое адресное пространство для параллельной программы — да, память является физически распределенной по различным частям системы, но логически разделяемой, так что пользователь видит единое адресное пространство. Система состоит из однородных базовых модулей (плат), состоящих из небольшого числа процессоров и блока памяти. Модули объединены с помощью высокоскоростного коммутатора. Поддерживается единое адресное пространство, аппаратно поддерживается доступ к удаленной памяти, т.е. к памяти других модулей. При этом доступ к локальной памяти осуществляется в несколько раз быстрее, чем к удаленной.

Кэш-память явилась причиной возникновения архитектуры NUMA — да, схема реализации компьютерной памяти, используемая в мультипроцессорных системах, когда время доступа к памяти определяется её расположением по отношению к процессору. Практически все архитектуры ЦПУ используют небольшое количество очень быстрой неразделяемой памяти, известной как кеш, который ускоряет обращение к часто требуемым данным.

Поиск команд, которые можно выполнять параллельно, в суперскалярных процессорах происходит во время работы программы — да, в суперскалярных компьютерах последовательность команд формируется динамически;

Параллелизм в классических VLIW-компьютерах выделяется компилятором — да, можно передать все планирование программному обеспечению, как это делается в конструкциях с VLIW. «Умный» компилятор должен выискать в программе все инструкции, которые являются совершенно независимыми, собрать их вместе в очень длинные строки (длинные инструкции) и затем отправить на одновременное исполнение функциональными модулями, количество которых, как минимум, не меньше, чем количество операций в такой длинной команде.

14. **Вопрос:** Отметьте правильные утверждения про компьютеры: классификация Флинна содержит 3 типа компьютеров; классификация Флинна содержит 4 типа компьютеров; классификация Флинна содержит 6 типов компьютеров; одним из признаков векторно-конвейерного компьютера является многопроцессорность; одним из признаков векторно-конвейерного компьютера является наличие хотя бы одного конвейера; одним из признаков векторно-конвейерного компьютера является наличие векторных регистров;

концепция неограниченного параллелизма при развитии компьютерной техники в отдалённом будущем может стать реальностью.

Ответ: Классификация Флинна содержит 3 типа компьютеров — нет.

Классификация Флинна содержит 4 типа компьютеров — да, Вычислительная система с одиночным потоком команд и одиночным потоком данных, Вычислительная система с одиночным потоком команд и множественным потоком данных, Вычислительная система со множественным потоком команд и одиночным потоком данных, Вычислительная система со множественным потоком команд и множественным потоком данных.

Классификация Флинна содержит 6 типов компьютеров — нет.

Одним из признаков векторно-конвейерного компьютера является многопроцессорность — нет.

Одним из признаков векторно-конвейерного компьютера является наличие хотя бы одного конвейера — да.

Одним из признаков векторно-конвейерного компьютера является наличие векторных регистров — да.

Концепция неограниченного параллелизма при развитии компьютерной техники в отдалённом будущем может стать реальностью — нет, эта концепция использует абстрактную идеальную параллельную ЭВМ, которая имеет бесконечно много параллельно работающих процессоров; все они работают синхронно под общим управлением и выполняют любую операцию точно и за одно и то же время; система имеет бесконечно большую память; все обмены информацией между процессорами и памятью, а также между самими процессорами осуществляются мгновенно и без конфликтов.

15. **Вопрос:** Какие из технологических этапов присущи только парадигме параллельного программирования: построение математической модели, декомпозиция, аранжировка, написание программы?

Ответ: Декомпозиция и аранжировка.

16. **Вопрос:** Чем декомпозиция по данным отличается от декомпозиции по вычислениям?

Ответ: Если в алгоритме сходным образом обрабатываются большие объёмы данных, то можно попробовать разделить эти _данные_ на части, каждая из которых допускает независимую обработку отдельным исполнителем. Это — декомпозиция по данным. Другой подход предполагает разделение _вычислений_ на зоны ответственности для их выполнения на разных исполнителях и определение данных, связанных с этими вычислениями. Это — декомпозиция по вычислениям (функциональная декомпозиция). Декомпозиция возможна не всегда. Существуют алгоритмы, которые принципиально не допускают при своей реализации участия нескольких исполнителей.

17. **Вопрос:** В чем заключаются достоинства и недостатки статического и динамического способов назначения задач виртуальным исполнителям?

Ответ:

Статический:	не требует вычислительной мощности(т.к. определяется на этапе компиляции) [+ , незначительный], проще в реализации [+ , незнач.], не адаптируется к конкретной вычислительной системе [-]
Динамический:	требует больше вычислительных ресурсов [- , незнач], сложнее в реализации [- , незнач], может адаптироваться к конкретной вычислительной системе [+]

18. **Вопрос:** Каковы основные цели этапа назначения: сокращение загрузки исполнителей, балансировка загрузки исполнителей, сокращение обменов данными между исполнителями, равномерный обмен данными между исполнителями, сокращение накладных расходов на назначение?

Ответ: Балансировка загрузки процессоров (ядер), уменьшение обменов данными между ними и сокращение накладных расходов на выполнение самого назначения.

19. **Вопрос:** Верно ли следующее утверждение: модель передачи сообщений в параллельном программировании применима только на системах с распределенной памятью? Обоснуйте свой ответ.

Ответ: Нет. Модель передачи сообщений применима на всех системах, но не всегда её применение разумно.

20. **Вопрос:** Приведите пример программного кода с внутренним параллелизмом, который, с Вашей точки зрения, нельзя распараллелить автоматически (если такие существуют).

Ответ:

```
do i = 1, N
    do j = 1, M
        a[i, j] = a[i-1, j-1] * 2
    enddo
enddo
```

21. **Вопрос:** В чем заключаются достоинства и недостатки различных подмоделей программирования в модели общей памяти?

Ответ: Предполагает, что приложение состоит из набора нитей исполнения (thread'ов), использующих разделяемые переменные и примитивы синхронизации. Выделяются две подмодели: явные нити исполнения — использование системных или библиотечных вызовов для организации работы thread'ов и программирование на языке высокого уровня с использованием соответствующих прагм. Первая подмодель обладает хорошей переносимостью, дает полный контроль над выполнением, но очень трудоемка. Вторая подмодель легка для программирования, но не дает возможности полностью контролировать решение задачи.

22. **Вопрос:** На каком этапе в параллельной программе задачи назначаются реальным физическим исполнителям: на этапе декомпозиции, на этапе назначения, на этапе аранжировки, на этапе отображения?

Ответ: на этапе отображения.

23. **Вопрос:** Современная парадигма параллельного программирования включает: 5, 6, 7, 8, 9, 10 этапов.

Ответ: 8.

24. **Вопрос:** Перечислите три основных принципа асимптотического анализа алгоритмов.

Ответ: 1. Оценка $T(n)$ не может быть привязана к конкретной вычислительной системе
2. Сравнение $T_A(n)$ и $T_B(n)$ на теоретической модели при малых значениях n возможно, но бесполезно.
3. Нас будет интересовать не сравнение собственно значений $T_A(n)$ и $T_B(n)$ при больших n , а сравнение их темпов роста

25. **Вопрос:** Пусть $T_1(n)$ и $T_2(n)$ — времена работы последовательных алгоритмов 1 и 2 соответственно. Если $T_1(n) = O(T_2(n))$, то алгоритм 1: лучше алгоритма 2, не хуже алгоритма 2, хуже алгоритма 2, ничего нельзя сказать.

Ответ: не хуже алгоритма 2.

26. **Вопрос:** Сформулируйте понятие оптимальности последовательного алгоритма.

Ответ: Если функция $f(n)$ является асимптотической нижней границей сложности принадлежащих классу A алгоритмов и если алгоритм $A \in A_{\text{такв}}$, что $T_A = O(f(n))$, то этот алгоритм оптимален в A по порядку сложности и $T_A = \Theta(f(n))$.

27. **Вопрос:** Какие из нижеперечисленных формулировок относятся к модели вычислительной системы RAM: время доступа к памяти одинаково для всех ячеек, независимо от того рассматривается операция чтения или операция записи; время выполнения всех операций на процессоре считается одинаковым; время выполнения основных операций на процессоре есть $\Theta(1)$?

Ответ: Время выполнения основных операций на процессоре есть $\Theta(1)$

28. **Вопрос:** Пусть два последовательных алгоритма решения одной задачи являются оптимальными по поведению. Что можно сказать о реальных временах решения при заданном параметре масштаба N : времена будут одинаковыми, ничего сказать нельзя?

Ответ: Ничего сказать нельзя

29. **Вопрос:** Пусть имеется два последовательных алгоритма решения одной и той же задачи — 1 и 2. Алгоритм 1 является оптимальным по поведению, алгоритм 2 не является оптимальным. Что можно сказать о реальных временах решения при заданном параметре масштаба N : время работы алгоритма 1 всегда будет меньше времени работы алгоритма 2, времена будут одинаковыми, ничего сказать нельзя?

Ответ: Ничего сказать нельзя.

30. **Вопрос:** Используя основную теорему асимптотического анализа, оцените асимптотическое поведение алгоритма сортировки слиянием. Является ли этот алгоритм оптимальным?

Ответ: Основная теорема асимптотического анализа:

Если $T(n) = aT(n/b) + f(n)$, где a и b — константы, $a \geq 1$, $b > 1$, $f(n) > 0$ и n принимает целые неотрицательные значения, то:

1. если $f(n) = O(n^{(\log_b a - \varepsilon)})$, где константа $\varepsilon > 0$, то $T(n) = \Theta(n^{(\log_b a)})$;

2. если $f(n) = \Theta(n^{(\log_b a)})$, то $T(n) = \Theta(n^{(\log_b a)} \log n)$;

3. если $f(n) = O(n^{(\log_b a + \varepsilon)})$, где константа $\varepsilon > 0$ и существуют константы c и N , $0 < c < 1$, $N > 0$, такие, что при $(n/b) > N$ выполнено $af(n/b) \leq cf(n)$, то $T(n) = \Theta(f(n))$.

Согласно 2 пункту теоремы асимптотического анализа, если $f(n) = \Theta(n^{(\log_b(a))})$, то $T(n) = \Theta(n^{(\log_b(a))} \ln(n))$, значит $T(n) = \Theta(n \ln(n))$. То, что $T(n) = \Theta(T_0(n))$,

начит оптимальность алгоритма. ($T_0(n)$ — теоретическая оценка времени решения задачи снизу).

31. **Вопрос:** Какие схемы разрешения конфликта при разрешенной одновременной записи различными исполнителями в одну ячейку памяти в модели вычислительной системы PRAM вы знаете?

Ответ: 1. Общая — запись разрешается только при всех одинаковых записываемых значениях.

2. Объединяющая — реально записывается значение некоторой функции от всех значений, которые пытались записать (max, min, среднее значение и т.д.).

3. Произвольная — записывается одно из предлагаемых значений, выбранное произвольно.

4. С приоритетами – у каждого исполнителя есть приоритет, запись произведет процессор с наибольшим приоритетом

Взято download.intuit.ru/video/618/1.ppt слайды 68-69

32. **Вопрос:** При вычислении теоретического значения ускорения для параллельных алгоритмов решения некоторой задачи на 4-х исполнителях были получены следующие значения: 2, 3.9, 5. Какие из значений являются корректными? Почему?

Ответ: Теоретическое Ускорение = (время работы наилучшего последовательного алгоритма при наихудших начальных данных) / (время работы параллельного алгоритма при тех же начальных данных).

5 не может быть. Так как если запустить параллельный алгоритм который даёт ускорение 5 на одном процессе, то ускорение на одном будет равняться 5/4, что больше одного, а значит наилучший последовательный алгоритм будет медленнее. Противоречие.

3.9 может быть. Например: распараллеливание `for(int i = 0; i < n; i++) sum += a[i];`

2 тоже может быть. Распараллеливание предыдущего цикла `for`, но параллелим на 2 процесса, а запускаем на 4. Или граф зависимостей параллелиться только на два подграфа.

33. **Вопрос:** При вычислении реального значения ускорения для параллельных алгоритмов решения некоторой задачи на 4-х исполнителях были получены следующие значения: 2, 3.9, 5. Какие из значений являются корректными? Почему?

Ответ: Практическое ускорение = (время работы последовательного алгоритма при наихудших начальных данных) / (время работы параллельной версии этого алгоритма при тех же начальных данных).

Все значение кроме 5 являются корректными. 2 т.к. просто граф зависимостей может не параллелиться (больше чем на 2 потока). 3.9 параллелим к примеру `for(int i = 0; i < n; i++) sum += a[i];`

5 не может быть. Так как если запустить параллельный алгоритм который даёт ускорение 5 на одном процессе, то ускорение будет равняться 5/4, что больше одного. Противоречие т.к должно быть меньше или равно 1 т.к алгоритм тот же.

34. **Вопрос:** Что означает высказывание, «некоторый параллельный алгоритм является оптимальным по стоимости»?

Ответ: Стоимость = (время работы параллельного алгоритма) × (количество исполнителей). Пусть время работы последовательного алгоритма $T_s(n) = \Theta(f(n))$, и стоимость параллельного алгоритма есть тоже $\Theta(f(n))$, тогда параллельный алгоритм называют *оптимальным по стоимости*.

(Взято из лекций Карпова 239)

35. **Вопрос:** Перечислите основные свойства графа алгоритма, реализованного программой.

Ответ:

1. Граф алгоритма всегда является ациклическим.
2. Граф алгоритма может быть мультиграфом.
3. Граф алгоритма является параметрическим.

(Взято из лекций Карпова 244)

36. **Вопрос:** Сколько строгих параллельных форм может существовать у графа алгоритма: 1, ограниченное количество, но больше 1; бесконечно много?

Ответ: стр 246

Дополнительная информация (это писать не надо):

“строгая параллельная форма у графа может быть не одна” (имеется в виду может быть больше 1) По идее может быть и 1, и ограниченное кол-во > 1 (некорректные варианты ответа?) ??? Но скорее всего имеется в виду **ограниченное количество, но больше 1**, так как в учебнике приведен пример > 1.

Итого: ограниченное количество, но больше 1

37. **Вопрос:** Докажите единственность канонической строгой параллельной формы графа алгоритма.

Ответ: стр 246

Дополнительная информация (это писать не надо):

*“Если в строгой параллельной форме существует вершина с индексом k , то длины всех путей, ведущих к этой вершине, очевидно, не превышают k . Путь к вершине графа с максимальной длиной назовем критическим путем. Среди множества строгих параллельных форм графа существует единственная параллельная форма, обладающая максимальной шириной ярусов и минимальной глубиной. Для этой строгой параллельной формы длина критического пути, ведущего к вершине с индексом k , всегда равна $k - 1$. Такую строгую параллельную форму принято называть **канонической параллельной формой**.”*

УТВЕРЖДЕНИЕ 2. Для любого ориентированного ациклического мультиграфа существует единственная каноническая параллельная форма.

Итого:

“Доказательство. Для доказательства существования опишем алгоритм построения канонической параллельной формы, а строгое доказательство ее единственности оставим вам для развлечения. В ациклическом графе выделим все вершины, в которые не входят дуги, и присвоим им индекс 1. Удалим эти вершины и все выходящие из них дуги из графа. В оставшемся графе выделим все вершины, не имеющие входящих дуг, и присвоим им индекс 2. Продолжим этот процесс, пока не исчерпаем все вершины графа. В полученной строгой параллельной форме вершине с индексом k индекс был присвоен на k -м шаге алгоритма. Это означает, что на k -м шаге не осталось ведущих к этой вершине дуг, в то время как на $k - 1$ -м шаге они были. Следовательно, длина критического пути в графе, ведущем к этой вершине, есть $k - 1$.”

Доказательства единственности не найдено.

38. **Вопрос:** Для графа алгоритма, содержащего 500 вершин, высота некоторой строгой параллельной формы равна 25. Что можно сказать о максимальном ускорении, которое может быть получено при реализации алгоритма на параллельной вычислительной системе в модели PRAM — оно будет всегда меньше 20, строго равно 20, может быть больше 20?

Ответ: стр 247

Дополнительная информация (это писать не надо):

“Если у вас есть параллельная вычислительная система с неограниченными возможностями, то вы можете организовать на ней реализацию алгоритма согласно канонической параллельной форме. При использовании модели PRAM время вычисления составит $\Theta(s)$, где s — глубина канонической параллельной формы, при этом вам потребуется N исполнителей, где N — максимальная ширина параллельных ярусов, а максимальное ускорение, которого вы сможете достигнуть, составит $\Theta(n)/\Theta(s) = \Theta(n/s)$ раз.”

Хз что такое высота строгой параллельной формы, и откуда берется ф-ла, но выглядит убедительно.

Итого: Доля последовательных вычислений (нижняя оценка) $B = n/N = 25/500 = 1/20$
Ускорение, получаемое при распараллеливании на s процессоров, можно оценить как $R = s/(s*B + (1 - B)) = 20*s/(s + 19) < 20$

39. **Вопрос:** Для программного текста, реализованного на Фортране 77

$$b(1) = a(1) * 2$$


```

do i = 2, 5
    b(i) = b(i-1) + a(i) * 2
enddo

```

постройте граф алгоритма, каноническую строгую параллельную форму и оцените максимально возможное ускорение при реализации алгоритма на параллельной вычислительной системе.

Ответ: стр 248:

Граф алгоритма:

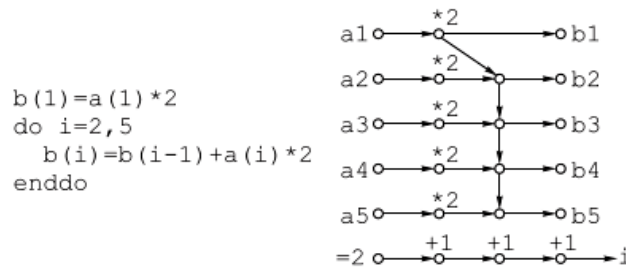


Рис. 5. Граф алгоритма, соответствующий программному фрагменту

Каноническую строгую параллельную форму построить по алгоритму из доказательства №37. “В ациклическом графе выделим все вершины, в которые не входят дуги, и присвоим им индекс 1. Удалим эти вершины и все выходящие из них дуги из графа. В оставшемся графе выделим все вершины, не имеющие входящих дуг, и присвоим им индекс 2. Продолжим этот процесс, пока не исчерпаем все вершины графа.”

Максимальное ускорение:

$$\text{Ускорение } R = 5 / (5 * 4/9 + (1 - 4/9)) = 9/5 \sim 2$$

4/9 – часть последовательного кода при правильном построении графа, 5 – разумное кол-во исполнителей.

40. **Вопрос:** Объясните понятия interleaving, недетерминированный и детерминированный набор активностей.

Ответ: Interleaving – явление, когда псевдопараллельное выполнение двух активностей приводит к чередованию их неделимых операций причём результат псевдопараллельного выполнения может отличаться от результата последовательного выполнения.

Набор активностей (например, программ) детерминирован, если всякий раз при псевдопараллельном исполнении для одного и того же набора входных данных он дает одинаковые выходные данные. В противном случае он недетерминирован.

41. **Вопрос:** Приведите пример набора активностей, для которого нарушены все условия Бернштейна, но который, тем не менее, является детерминированным.

Ответ: Справка:

Набор входных переменных программы $R(P)$ суть объединение наборов входных переменных для всех ее неделимых действий. Набор выходных переменных программы $W(P)$ суть объединение наборов выходных переменных для всех ее неделимых действий.

Достаточные условия Бернштейна.

Если для двух данных активностей P и Q

- пересечение $W(P)$ и $W(Q)$ пусто,
- пересечение $W(P)$ с $R(Q)$ пусто,

- пересечение $R(P)$ и $W(Q)$ пусто,
тогда выполнение P и Q детерминировано.

Итого:

$S1: x = x/2;$

$S2: x = x*2;$

42. **Вопрос:** Если в программе встречаются два оператора, приведенных в динамическом порядке следования,

$a = b+c;$

$d = e+f;$

то можно ли их исполнять параллельно? Обоснуйте свое утверждение.

Ответ: Да, можно. Они удовлетворяют условиям Бернштейна, а значит они детерминированы, а значит вне зависимости от последовательности их выполнения результат будет одинаков. Если мы их выполним параллельно, результат будет тем же, что и при последовательном выполнении.

43. **Вопрос:** Перечислите виды зависимостей между операторами (блоками операторов), которые Вам известны.

Ответ: Зависимость по выходным данным (нарушение первого условия Бернсайда).

Антизависимость (нарушение третьего условия Бернсайда).

Истинная(потокосная) зависимость (нарушение второго условия Бернсайда).

Зависимость по управлению - условные операторы, изменяющие динамический порядок других операторов при различных наборах входных данных.

Зависимость по ресурсам. Если существуют два оператора, выполняющих деление и не имеющих зависимости по данным и существует единственный исполнитель, умеющий делить одно данное на другое, то параллельное исполнение операторов оказывается невозможным.

44. **Вопрос:** Какие из видов зависимостей по данным могут являться принципиальным препятствием к распараллеливанию: истинная зависимость, зависимость по выходным данным, антизависимость?

Ответ: Истинная зависимость. Так как второй оператор использует результат выполнения первого. В случае зависимости по выходным данным достаточно переименовать выходные переменные. Если есть антизависимость, можно сделать копию входных данных в локальную память первого исполнителя до начала исполнения обоих операторов.

45. **Вопрос:** Постройте граф зависимостей по данным между операторами для фрагмента текста программы, расположенного в динамическом порядке следования:

$S1: x = e + 2z$

$S2: y = 2f + x$

$S3: z = z + y$

$S4: y = z + x$

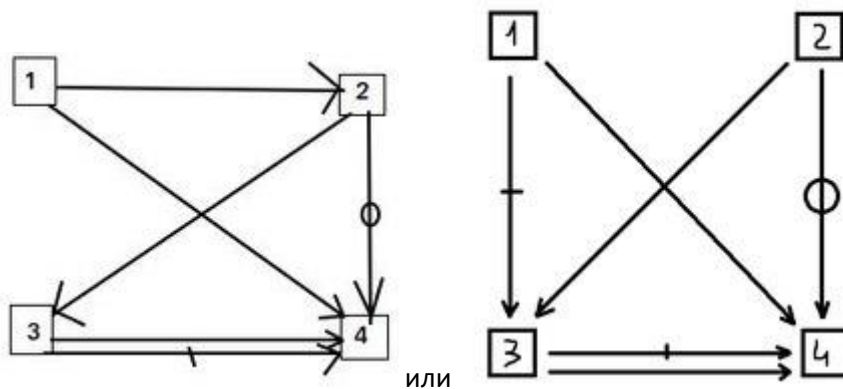
Ответ: Вспоминаем условия Бернштейна:

$W(1) \&\& W(2) = 0$, иначе – зависимость по выходу (линия с окружностью на графе);

$W(1) \&\& R(2) = 0$, иначе – истинная зависимость (обычная линия на графе);

$R(1) \&\& W(2) = 0$, иначе – антизависимость (перечёркнутая линия на графе);

где $R(n1)$ и $W(n2)$ означают соответственно множество входных (read) и выходных (write) данных для процессоров с номерами $n1$ и $n2$.



46. **Вопрос:** Постройте граф зависимостей по данным между операторами для фрагмента текста программы, расположенного в динамическом порядке следования:

S1: $x = x + 2y$

S2: $y = 2f + x$

S3: $z = z + z$

S4: $y = z + x$

Ответ: Вспоминаем условия Бернштейна:

$W(1) \&\& W(2) = 0$, иначе – зависимость по выходу (линия с окружностью на графе, тип A);

$W(1) \&\& R(2) = 0$, иначе – истинная зависимость (обычная линия на графе, тип B);

$R(1) \&\& W(2) = 0$, иначе – антизависимость (перечеркнутая линия на графе, тип C);

где $R(n1)$ и $W(n2)$ означают соответственно множество входных (read) и выходных (write) данных для процессоров с номерами $n1$ и $n2$.

В итоге применяя эти правила рисуем граф(опишу его словами, т.к. нарисовать проблема, в виде № вершины, из которой идет стрелка - № вершины, куда идет стрелка - ее тип, 2 вершины может соединять несколько дуг-это не ошибка!)

S1 – S2 – C

S1 – S2 – B

S1 – S4 – B

S1 – S4 – C

S2 – S4 – A

S3 – S4 – B

47. **Вопрос:** Для цикла

do $i = 1, u$

S₁: $a(i) = d(i) + 5 * i$

S₂: $c(i) = a(i-1) * 2$

enddo

определите расстояние зависимости, тип зависимости и возможность распараллеливания.

Ответ: Развернем цикл на несколько итераций

S11: $a[1] = d[1] + 5 * 1$

S21: $c[1] = a[0] * 2$

S12: $a[2] = d[2] + 5 * 2$

S22: $c[2] = a[1] * 2$

S22 – сток, S11 – источник $\Rightarrow a = 2, k = 1 \Rightarrow$ расстояние зависимости $D = a - k = 1 > 0 \Rightarrow$ имеем истинную зависимость, которую нельзя распараллелить

48. **Вопрос:** Для цикла

```
do i = 1, u
  S1:   a(i) = d(i) + 5*i
  S2:   c(i) = a(i+1)*2
enddo
```

определите расстояние зависимости, тип зависимости и возможность распараллеливания.

Ответ: Развернем цикл на несколько итераций

```
S11: a[1] = d[1] + 5*1
S21: c[1] = a[2]*2
S12: a[2] = d[2] + 5*2
S22: c[2] = a[3]*2
```

S21 – сток, S12 – источник $\Rightarrow a = 1, k = 2 \Rightarrow$ расстояние зависимости $D = a - k = -1 < 0 \Rightarrow$ имеем антизависимость, которую можно распараллелить при помощи дублирования перед выполнением цикла входных данных на u исполнителей, получив при этом ускорение $\sim u$.

49. **Вопрос:** Для цикла

```
do i = 1, u
  S1:   a(i) = d(i) + 5*i
  S2:   c(i) = a(i)*2
enddo
```

определите расстояние зависимости, тип зависимости и возможность распараллеливания.

Ответ: Развернем цикл на несколько итераций

```
S11: a[1] = d[1] + 5*1
S21: c[1] = a[1]*2
S12: a[2] = d[2] + 5*2
S22: c[2] = a[2]*2
```

S21 – сток, S11 – источник $\Rightarrow a = 1, k = 1 \Rightarrow$ расстояние зависимости $D = a - k = 0 \Rightarrow$ имеем потоковую зависимость, которую можно распараллелить на u исполнителей (дав каждому исполнителю уникальный номер итерации на выполнение), получив при этом ускорение $\sim u$.

50. **Вопрос:** С чем связаны основные проблемы определения вектора расстояний при анализе вложенных циклов на зависимость по данным?

Ответ: точного ответа нет. Предполагаемый:

Для нахождения вектора расстояний необходимо решить систему диофантовых уравнений, которые в общем случае могут быть нерешаемы.

51. **Вопрос:** Для цикла

```
do j1 = 1, u1
  do j2 = 1, u2
    S1: a(i, j) = b(i, j) * 2
    S2: c(i, j) = a(i, j - 1) + 1
  enddo
enddo
```

определите вектор расстояний, вектор направлений и тип зависимости.

Ответ: Развернём цикл на несколько итераций

```
S1(1,1): a[1, 1] = b[1, 1]*2
```

S2(1,1): $c[1, 1] = a[1, 0] + 1$

S1(1,2): $a[1, 2] = b[1, 2] * 2$

S2(1,2): $c[1, 2] = a[1, 1] + 1$

...

S1(2,1): $a[2, 1] = b[2, 1] * 2$

S2(2,1): $c[2, 1] = a[2, 0] + 1$

...

S2(1,2) – сток, S1(1,1) – источник $\Rightarrow A = (1, 2), K = (1, 1) \Rightarrow$ вектор расстояния $D = A - K = (0, 1)$
 $\Rightarrow$ вектор направлений $d = (=, <)$. $a[1, 1]$ сначала вычисляется, а затем используется, тогда имеем истинную зависимость.

52. **Вопрос:** Для цикла

```
do j1 = 1,u1
```

```
do j2 = 1,u2
```

```
S1: a(i, j) = b(i, j) * 2
```

```
S2: c(i, j) = a(i, j + 2) + 1
```

```
enddo
```

```
enddo
```

определите вектор расстояний, вектор направлений и тип зависимости.

Ответ: Развернём цикл на несколько итераций

S1(1,1): $a[1, 1] = b[1, 1] * 2$

S2(1,1): $c[1, 1] = a[1, 3] + 1$

S1(1,2): $a[1, 2] = b[1, 2] * 2$

S2(1,2): $c[1, 2] = a[1, 4] + 1$

S1(1,3): $a[1, 3] = b[1, 3] * 2$

S2(1,3): $c[1, 3] = a[1, 5] + 1$

...

S1(2,1): $a[2, 1] = b[2, 1] * 2$

S2(2,1): $c[2, 1] = a[2, 3] + 1$

...

S2(1,1) – сток, S1(1,3) – источник $\Rightarrow A = (1, 1), K = (1, 3) \Rightarrow$ вектор расстояния $D = A - K = (0, -2)$
 $\Rightarrow$ вектор направлений $d = (=, >)$. $a[1, 3]$ сначала используется, а затем вычисляется, тогда имеем антизависимость.

53. **Вопрос:** Для цикла

```
do j1 = 1,u1
```

```
do j2 = 1,u2
```

```
S1: a(i, j) = b(i, j) * 2
```

```
S2: c(i, j) = a(i - 1, j + 2) + 1
```

```
enddo
```

```
enddo
```

определите вектор расстояний, вектор направлений и тип зависимости.

Ответ: Развернём цикл на несколько итераций

S1(1,1): $a[1, 1] = b[1, 1] * 2$

S2(1,1): $c[1, 1] = a[0, 3] + 1$

S1(1,2): $a[1, 2] = b[1, 2] * 2$

S2(1,2): $c[1, 2] = a[0, 4] + 1$

S1(1,3): $a[1, 3] = b[1, 3] * 2$

S2(1,3): $c[1, 3] = a[0, 5] + 1$

...

S1(2,1): $a[2, 1] = b[2, 1] * 2$

S2(2,1): $c[2, 1] = a[1, 3] + 1$

...

S2(2,1) – сток, S1(1,3) – источник $\Rightarrow A = (2, 1), K = (1, 3) \Rightarrow$ вектор расстояния $D = A - K = (1, -2)$
 $\Rightarrow$ вектор направлений $d = (<, >)$. $a[1, 3]$ сначала вычисляется, а затем используется, тогда имеем истинную зависимость.

54. **Вопрос:** Для цикла

do j1 = 1,u1

do j2 = 1,u2

S₁: $a(i, j) = b(i, j) * 2$

S₂: $c(i, j) = a(i + 1, j + 3) + 1$

enddo

enddo

определите вектор расстояний, вектор направлений и тип зависимости.

Ответ: Развернём цикл на несколько итераций

S1(1,1): $a[1, 1] = b[1, 1] * 2$

S2(1,1): $c[1, 1] = a[2, 4] + 1$

S1(1,2): $a[1, 2] = b[1, 2] * 2$

S2(1,2): $c[1, 2] = a[2, 5] + 1$

S1(1,3): $a[1, 3] = b[1, 3] * 2$

S2(1,3): $c[1, 3] = a[2, 6] + 1$

S1(1,4): $a[1, 4] = b[1, 4] * 2$

S2(1,4): $c[1, 4] = a[2, 7] + 1$

...

S1(2,1): $a[2, 1] = b[2, 1] * 2$

S2(2,1): $c[2, 1] = a[3, 3] + 1$

S1(2,2): $a[2, 2] = b[2, 2] * 2$

S2(2,2): $c[2, 2] = a[3, 5] + 1$

S1(2,3): $a[2, 3] = b[2, 3] * 2$

S2(2,3): $c[2, 3] = a[3, 6] + 1$

S1(2,4): $a[2, 4] = b[2, 4] * 2$

S2(2,4): $c[2, 4] = a[3, 7] + 1$

...

S2(1,1) – сток, S1(2,4) – источник $\Rightarrow A = (1, 1), K = (2, 4) \Rightarrow$ вектор расстояния $D = A - K = (-1, -3) \Rightarrow$ вектор направлений $d = (>, >)$. $a[2, 4]$ сначала используется, а затем вычисляется, тогда имеем антизависимость.

55. **Вопрос:** Для цикла

do j1 = 1,u1

do j2 = 1,u2

S₁: $a(i, j) = b(i, j) * 2$

S₂: $c(i, j) = a(i, j + 2) + 1$

enddo

enddo

определите вектор расстояний, вектор направлений и тип зависимости.

Ответ: Развернём цикл на несколько итераций

S1(1,1): $a[1, 1] = b[1, 1] * 2$

S2(1,1): $c[1, 1] = a[1, 3] + 1$

S1(1,2): $a[1, 2] = b[1, 2]*2$

S2(1,2): $c[1, 2] = a[1, 4] + 1$

S1(1,3): $a[1, 3] = b[1, 3]*2$

S2(1,3): $c[1, 3] = a[1, 5] + 1$

...

S1(2,1): $a[2, 1] = b[2, 1]*2$

S2(2,1): $c[2, 1] = a[2, 3] + 1$

...

S2(1,1) – сток, S1(1,3) – источник $\Rightarrow A = (1, 1)$, $K = (1, 3) \Rightarrow$ вектор расстояния $D = A - K = (0, -2)$
 $\Rightarrow$ вектор направлений $d = (=, >)$. $a[1, 3]$ сначала используется, а затем вычисляется, тогда имеем антизависимость.

56. **Вопрос:** Для цикла

```
do j1 = 1,u1
```

```
do j2 = 1,u2
```

```
S1: a(i, j) = b(i, j) * 2
```

```
S2: c(i, j) = a(i, j) + 1
```

```
enddo
```

```
enddo
```

определите вектор расстояний, вектор направлений и тип зависимости.

Ответ: Развернём цикл на несколько итераций

S1(1,1): $a[1, 1] = b[1, 1]*2$

S2(1,1): $c[1, 1] = a[1, 1] + 1$

S1(1,2): $a[1, 2] = b[1, 2]*2$

S2(1,2): $c[1, 2] = a[1, 2] + 1$

...

S2(1,1) – сток, S1(1,1) – источник $\Rightarrow A = (1, 1)$, $K = (1, 1) \Rightarrow$ вектор расстояния $D = A - K = (0, 0)$
 $\Rightarrow$ вектор направлений $d = (=, =)$, значения сначала вычисляются, а затем используются
 $\Rightarrow$ истинная зависимость.

57. **Вопрос:** Для цикла

```
do j1 = 1,u1
```

```
do j2 = 1,u2
```

```
S1: a(i, j) = b(i, j) * 2
```

```
S2: c(i, j) = a(i + 2, j - 3) + 1
```

```
enddo
```

```
enddo
```

определите вектор расстояний, вектор направлений и тип зависимости.

Ответ: Развернём цикл на несколько итераций

S1(1,1): $a[1, 1] = b[1, 1]*2$

S2(1,1): $c[1, 1] = a[3, -2] + 1$

S1(1,2): $a[1, 2] = b[1, 2]*2$

S2(1,2): $c[1, 2] = a[3, -1] + 1$

S1(1,3): $a[1, 3] = b[1, 3]*2$

S2(1,3): $c[1, 3] = a[3, 0] + 1$

S1(1,4): $a[1, 3] = b[1, 4]*2$

S2(1,4): $c[1, 3] = a[3, 1] + 1$

...

$S1(2,1): a[2, 1] = b[2, 1]*2$
 $S2(2,1): c[2, 1] = a[4, -2] + 1$
 $S1(2,2): a[2, 2] = b[2, 2]*2$
 $S2(2,2): c[2, 2] = a[4, -1] + 1$
 $S1(2,3): a[2, 3] = b[2, 3]*2$
 $S2(2,3): c[2, 3] = a[4, 0] + 1$
 $S1(2,4): a[2, 4] = b[2, 4]*2$
 $S2(2,4): c[2, 4] = a[4, 1] + 1$

...

$S1(3,1): a[3, 1] = b[2, 1]*2$
 $S2(3,1): c[3, 1] = a[5, -2] + 1$
 $S1(3,2): a[3, 2] = b[2, 2]*2$
 $S2(3,2): c[3, 2] = a[5, -1] + 1$
 $S1(3,3): a[3, 3] = b[2, 3]*2$
 $S2(3,3): c[3, 3] = a[5, 0] + 1$
 $S1(3,4): a[3, 4] = b[2, 4]*2$
 $S2(3,4): c[3, 4] = a[5, 1] + 1$

...

$S2(1,4)$ – сток, $S1(3,1)$ – источник $\Rightarrow A = (1, 4)$, $K = (3, 1) \Rightarrow$ вектор расстояния $D = A - K = (-2, 3) \Rightarrow$ вектор направлений $d = (>, <)$, значения сначала используются, а потом заново вычисляются $\Rightarrow$ антизависимость.

58. **Вопрос:** Пусть вектор направлений для двумерного цикла имеет вид («=»,«=»). Какие из следующих утверждений являются верными: цикл может быть распараллелен по внешнему индексу безо всяких ограничений, по внешнему индексу при условии дублирования входных данных, по внутреннему индексу при условии барьерной синхронизации между итерациями внешнего цикла, по двум индексам без ограничений, не может быть распараллелен?

Ответ: Пусть вектор направлений для двумерного цикла имеет вид («=»,«=»). Верные: цикл может быть распараллелен по внешнему индексу безо всяких ограничений, по двум индексам без ограничений(из этого следует, что наложив любые условия, например барьерной синхронизации, цикл можно распараллелить), тогда добавим: по внешнему индексу при условии дублирования входных данных, по внутреннему индексу при условии барьерной синхронизации между итерациями внешнего цикла.

59. **Вопрос:** Пусть вектор направлений для двумерного цикла имеет вид («<»,«=»). Какие из следующих утверждений являются верными: цикл может быть распараллелен по внешнему индексу безо всяких ограничений, по внешнему индексу при условии дублирования входных данных, по внутреннему индексу при условии барьерной синхронизации между итерациями внешнего цикла, по двум индексам без ограничений, не может быть распараллелен?

Ответ: Пусть вектор направлений для двумерного цикла имеет вид («<»,«=»). Верные: По идее, только по внутреннему циклу без ограничений. Но такого варианта нет. Тогда берем: по внутреннему индексу при условии барьерной синхронизации между итерациями внешнего цикла.

60. **Вопрос:** Пусть вектор направлений для двумерного цикла имеет вид («<»,«<»). Какие из следующих утверждений являются верными: цикл может быть распараллелен по внешнему индексу безо всяких ограничений, по внешнему индексу при условии дублирования входных данных, по внутреннему индексу при условии барьерной

Ответ: Пусть вектор направлений для двумерного цикла имеет вид («<», «<»). Верные: По внутреннему индексу при условии барьерной синхронизации между итерациями внешнего цикла.

- Ответ:** смотри таблицу после 66-ого вопроса.

- Ответ:** смотри таблицу после 66-ого вопроса.

- Ответ:** смотри таблицу после 66-ого вопроса.

- Ответ:** смотри таблицу после 66-ого вопроса.

- Ответ:** смотри таблицу после 66-ого вопроса.

66. **Вопрос:** Пусть вектор направлений для двумерного цикла имеет вид («>»,«<»). Какие из следующих утверждений являются верными: цикл может быть распараллелен по внешнему индексу безо всяких ограничений, по внешнему индексу при условии дублирования входных данных, по внутреннему индексу при условии барьерной синхронизации между итерациями внешнего цикла, по двум индексам без ограничений, не может быть распараллелен?

Ответ: Во-первых, цикл может быть распараллелен по внешнему индексу при условии дублировании входных данных. Во-вторых, цикл может быть распараллелен без ограничений по внутреннему индексу, а значит и по внутреннему индексу при условии барьерной синхронизации между итерациями внешнего цикла.

58-66 Пусть вектор направлений для двумерного цикла имеет вид (A, B). Какие из следующих утверждений являются верными:

цикл может быть распараллелен

1 по внешнему индексу безо всяких ограничений,

2 по внешнему индексу при условии дублирования входных данных,

3 по внутреннему индексу при условии барьерной синхронизации между итерациями внешнего цикла,

4 по двум индексам без ограничений,

5 не может быть распараллелен?

	58 ==	59 <=	60 <<	61 =<	62 >=	63 =>	64 >>	65 <>	66 ><
1	+			+		+			
2					+		+		+
3		+	+					+	
4	+								
5									

67. **Вопрос:** Пусть для некоторого многомерного цикла вектор направлений имеет вид («=», «<», ..., «>»). Что можно сказать про тип зависимости, обнаруживаемой между операторами после раскрытия цикла: это — истинная зависимость, это — антизависимость, ничего сказать нельзя?

Ответ: Согласно главному утверждению (Карпов, стр. 265) в цикле присутствует истинная зависимость.

68. **Вопрос:** Пусть для некоторого многомерного цикла вектор направлений имеет вид («>», «<», ..., «>»). Что можно сказать про тип зависимости, обнаруживаемой между операторами после раскрытия цикла: это — истинная зависимость, это — антизависимость, ничего сказать нельзя?

Ответ: Ничего сказать нельзя (вариант Серебрякова Кости) или же антизависимость (вариант Пушнякова Лёши).

69. **Вопрос:** Пусть для некоторого многомерного цикла вектор направлений имеет вид («<», «=», ..., «>»). Что можно сказать про тип зависимости, обнаруживаемой между операторами после раскрытия цикла: это — истинная зависимость, это — антизависимость, ничего сказать нельзя?

Ответ: Согласно главному утверждению (Карпов, стр. 265) в цикле присутствует истинная зависимость.

70. **Вопрос:** Эквивалентное преобразование программы — это: преобразование динамического порядка следования операторов, сохраняющее результат при любых входных данных; преобразование, сохраняющее граф алгоритма; преобразование, сохраняющее зависимости по данным между операторами, ничего из вышеперечисленного.

Ответ: Эквивалентным преобразованием последовательной программы является изменение динамического порядка её операторов, сохраняющее граф алгоритма. Поэтому как ответ на вопрос следует выбрать — ничего из вышеперечисленного.

71. **Вопрос:** Почему знание о возможности эквивалентного изменения порядка вложенности во вложенных циклах для распараллеливания программ является важным?

Ответ: (ответ не найден :()

Карпов везде пишет, что это очень важно, что мы можем поменять местами внешний и внутренний циклы (где это возможно без изменения вычислений). Почему это так — не ясно. Как именно это влияет на распараллеливание — не ясно. Возможно, тут как-то стоит провести параллель с изменением порядка вложенности циклов при обходе матрицы. Если поменять местами циклы (то есть сначала провести цикл по j , а потом уже по i , а не наоборот, как мы всегда делаем), то программа затратит много больше времени, так как курсор будет двигаться не по порядку, а прыгая из начала в конец.

Цитата со стр. 266, вдруг пригодится: «Для произвольного цикла возможно распараллеливание по любому индексу, соответствующему компоненту «=» в векторе направлений. Уровень вложенности, соответствующий этому компоненту, можно поменять местами с любым соседним уровнем вложенности с сохранением результата вычислений. Два соседних уровня вложенности, которым соответствуют одинаковые компоненты вектора направлений, также можно поменять местами...»

Примеры с изменением порядка вложенности до стр.266.

72. **Вопрос:** Перечислите дополнительные условия в теореме о достаточности приемов разделения цикла, выравнивания цикла и допустимой перестановки операторов в теле цикла для устранения зависимостей, связанных с циклом.

Ответ: (Карпов, стр.270)

Приемы разделения цикла, выравнивания цикла и допустимая перестановка операторов в теле цикла достаточны для устранения зависимостей, связанных с циклом, если:

1. в цикле нет рекурсивных зависимостей;
2. расстояние для каждой зависимости есть константа, не зависящая от индекса цикла.

73. **Вопрос:** Что означает выражение: в цикле нет рекурсивных зависимостей по данным?

Ответ: (Карпов, стр.268)

Два набора операторов в цикле могут быть распределены по двум циклам, если в исходном цикле не существует циклов истинной зависимости между операторами, т. е. нет рекурсивных зависимостей.

74. **Вопрос:** Справедливо ли утверждение, если между операторами цикла существует истинная зависимость — цикл по итерациям распараллелить невозможно? Обоснуйте ответ.

Ответ: (Карпов, стр.257. Если в вопросе появится что-то именно для вложенных циклов, то это можно посмотреть на стр. 266)

Нет, утверждение не является справедливым. Пример цикла с истинной зависимостью, для которого возможно распараллеливание (для $D=2$):

do $i=1,u$

S1: $a[i]=d[i]+5*i$;

S2: $c[i]=a[i-2]*2$;

enddo

Развернем цикл:

S11: $a[1]=d[1]+5*1$;

S21: $c[1]=a[-1]*2$;

S12: $a[2]=d[2]+5*2$;

S22: $c[2]=a[0]*2$;

S13: $a[3]=d[3]+5*3$;

S23: $c[3]=a[1]*2$;

...

Истинная зависимость существует, так как при последовательном выполнении операторов $a[1]$ сначала изменили, а потом использовали. Но: заметим, что значение, полученное на первой итерации, используется лишь в третьей итерации, а значение, полученное на третьей итерации — лишь в пятой и т. д. То есть выполнение итераций возможно на двух носителях, один из которых реализует все четные итерации, а другой нечетные.

Утверждение: при $D>1$ цикл может быть распараллелен не более, чем на D носителей.

75. **Вопрос:** Справедливо ли утверждение, если между операторами невложенного цикла существует истинная зависимость с расстоянием зависимости равным 1 — цикл по итерациям распараллелить принципиально невозможно? Обоснуйте ответ.

Ответ: (Карпов, стр.268)

Утверждение справедливо.

Рассмотрим пример цикла с $D=1$:

do $i=1, u$

 S1: $a[i]=d[i]+5*i$;

 S2: $c[i]=a[i-1]*2$;

enddo

Развернем цикл:

S11: $a[1]=d[1]+5*1$;

S21: $c[1]=a[0]*2$;

S12: $a[2]=d[2]+5*2$;

S22: $c[2]=a[1]*2$;

...

Заметим, что при последовательном выполнении операторов значение $a[1]$ сначала изменяется, а потом используется, т. е. истинная зависимость. Выполнение операций параллельно невозможно, так как каждая следующая итерация обращается к предыдущей, и «разделить» их никак нельзя.

Аналогичный вывод для других циклов, где $D=1$.

(После эквивалентных преобразований программы, к примеру, разделения цикла, распараллеливание полученных циклов уже становится возможно)

76. **Вопрос:** Какие приемы распараллеливания циклов при наличии зависимости операторов цикла по скалярным переменным являются эквивалентным преобразованием программ: приватизация, «ликвидация» индукционных переменных, редукция?

Ответ: Только приватизация, так как ликвидация индукционных переменных и редукция меняют граф алгоритма.

77. **Вопрос:** Справедливо ли следующее утверждение: распараллеливание с помощью приема редукции всегда является неэквивалентным преобразованием программы? Обоснуйте свой ответ.

Ответ: Да, оно справедливо. При редукции мы распараллеливаем какую-либо редукционную операцию (например, сложение) на несколько вычислителей. Соответственно меняется порядок операций и, значит, граф алгоритма.

Пример:

$j = 0$

do $i = 1, u$

$j = j + a[i]$

enddo

Если нам требуется посчитать сумму восьми элементов массива, то это можно сделать так: Посчитать парные суммы $a_{12} = a[1] + a[2]$, . . . , $a_{78} = a[7] + a[8]$ на четырех процессорах, затем суммы $a_{1234} = a_{12} + a_{34}$, $a_{5678} = a_{56} + a_{78}$ – на двух процессорах и после этого – окончательную сумму.

Это лишь пример, строгого доказательства нету.

78. **Вопрос:** Справедливо ли следующее утверждение: распараллеливание с помощью приема «ликвидации» индукционных переменных всегда является неэквивалентным преобразованием программы? Обоснуйте свой ответ.

Ответ: Это справедливое рассуждение.

Пример:

```
j = 0
do i = 1, u
  j = j + k
  a[i] = j
enddo
```

Здесь между всеми итерациями существует истинная зависимость по j. Можно заметить, что с точки зрения математики (без взгляда на остальную программу), что на n-ой итерации значение $j = k * n$. Так что можно (с этой точки зрения) написать:

```
do i = 1, u
  j = k * i
  a[i] = j
enddo
```

Получаем параллельный цикл. Но мы изменяем граф алгоритма (неэквивалентное преобразование.)

79. **Вопрос:** Можно ли устранить в последовательной программе истинные зависимости по данным для получения корректной параллельной версии?

Ответ: Можно, используя ликвидацию индукционных переменных и редукцию. Но при этом необходимо быть осторожным, т.к. меняется граф алгоритма.

80. **Вопрос:** Какие схемы распределения работ на этапе назначения между виртуальными исполнителями Вам известны?

Ответ: Статическая (каждому исполнителю фиксированный набор задач), динамическая (каждому исполнителю по 1 задаче из очереди задач; после выполнения берется следующая задача), смешанная (динамическая, но задач берется несколько)).

81. **Вопрос:** Система ОДУ не является жесткой и решается явным методом Рунге-Кутты. Будет ли реализованная для такого уравнения последовательная программа эффективно распараллеливаться?

Ответ: Метод Рунге-Кутты (например, 4 порядка) для решения системы дифференциальных уравнений вида

$$y' = f(t, y)$$

задаётся уравнениями

$$y_{n+1} = y_n + 1/6 h (k_1 + k_2 + k_3 + k_4),$$

где выражения k_1, k_2, k_3, k_4 задаются формулами

$$\begin{aligned} k_1 &= f(t_n, y_n), \quad k_2 = f(t_n + 0.5h, y_n + 0.5h * k_1) \\ k_3 &= f(t_n + 0.5h, y_n + 0.5h * k_2), \quad k_4 = f(t_n + h, y_n + h * k_3) \end{aligned}$$

Составляя граф алгоритма, видим, что он имеет большое количество ярусов, поэтому такая версия алгоритма не допускает эффективного распараллеливания.

82. **Вопрос:** Жесткая система обыкновенных дифференциальных уравнений решается W-методом с заменой точного обращения матрицы приближенным по методу Шульца. Какая версия метода допускает более эффективную параллельную реализацию – с несколькими итерациями для вычисления матрицы или с учетом нескольких последовательных степеней невязки?

Ответ: W-метод заключается в следующем.

Рассмотрим систему дифференциальных уравнений вида

$$y' = F(x, y)$$

Формула для вычисления решения W-методом имеет вид

$$(E - h * d_{ii} A) k_i = F(y_n + h * \sum_{j=1}^{i-1} a_{ij} k_j) + h A \sum_{j=1}^{i-1} d_{ij} k_j,$$

где A – некоторая произвольная матрица, такая, что $(E - h * d_{ii} A)$ обратима.

При этом очередной шаг решения вычисляется по формуле

$$y_{n+1} = y_n + h * \sum_{i=1}^s b_i * k_i$$

Метод Шульца заключается в замене обращения матрицы использованием последовательной частичной суммы для ряда $(E - h * d_{ii} A)^{-1}$

Кроме итерационной процедуры обращения матрицы использование частичной суммы ряда по степеням невязки

$$B_i^{n+1} = \left(E + R_i^{n+1} + (R_i^{n+1})^2 + \dots + (R_i^{n+1})^{s+1} \right) B_i^n$$

должно формально приводить к лучшим результатам для вычисления значения обратной матрицы.

Формально и такой способ вычисления не содержит нераспараллеливаемых операций – только перемножения и сложения матриц. Но с точки зрения организации вычислений имеется принципиальная разница. Если для реализации нескольких итераций у нас не меняется матрица $E + R_i^{n+1}$, соответствующие значения могут быть разосланы исполнителям в начале процесса вычислений и не будут меняться в ходе итераций, то при вычислении матрицы суммы степенного ряда нам необходимо сохранять все промежуточные результаты. Возрастает количество обращений к памяти и объем пересылаемой информации.

То есть лучше способ с несколькими итерациями.

83. **Вопрос:** Будет ли однократно диагонально неявный метод, основанный на методе Розенброка, более эффективным с точки зрения распараллеливания по сравнению со всеми остальными методами того же класса?

Ответ: Метод Розенброка. Пусть имеется жесткая система дифференциальных уравнений

$$y' = f(y, t)$$

она заменяется разностной формой типа

$$(E - a \Delta A - b \Delta^2 A^2) * (y_{i+1} - y_i) / \Delta = f(y_i + c * \Delta * f(y_i))$$

где Δ — шаг интегрирования, а a, b, c — неизвестные параметры. Если матрица A диагональная, то обращение матрицы $(E - a \Delta A - b \Delta^2 A^2)$ сводится к обращению диагональной матрицы. Если функция $f(\cdot)$ по координатам независима, то метод допускает эффективное распараллеливание. В противном случае формулы имеют вид

$$y_{i+1} = y_i + X^{-1} * f(\dots),$$

где X – некоторая матрица. Если вычисление f нельзя распараллелить, то замена матрицы на диагональную не приносит выигрыша..

84. **Вопрос:** Метод параллельной стрельбы будет более эффективным по сравнению с методом стрельбы: А. Если число исполнителей превышает количество добавленных

систем линейных дифференциальных уравнений в вариациях Б. В любом случае, так как быстрее сходятся итерации по пристрелочным параметрам В. Если исходная краевая задача хорошо обусловлена, и в этом случае быстрее сходятся итерации по пристрелочным параметрам

Ответ: Метод параллельной стрельбы будет более эффективным по сравнению с методом стрельбы: А. Если число исполнителей превышает количество добавленных систем линейных дифференциальных уравнений в вариациях Б. В любом случае, так как быстрее сходятся итерации по пристрелочным параметрам В. Если исходная краевая задача хорошо обусловлена, и в этом случае быстрее сходятся итерации по пристрелочным параметрам

пруфлинк. http://www.nsc.ru/rus/textbooks/akhmerov/nm-ode_unicode/2-2.html

Метод стрельбы заключается в следующем. Имеется некоторая задача с краевыми условиями, например диффур 2 порядка с условиями на концах. Такая задача сводится с помощью метода стрельбы к решению задач Коши.

Мы забываем про правое граничное условие, и подбираем производную на левом конце так, чтобы решение справа совпало с условием. Параметр (значение производной на левом конце) ищется с некоторой точностью, например, сеточным методом.

Метод параллельной стрельбы решает все эти задачи Коши одновременно. Я думаю, что здесь правильный ответ Б, то есть параллельный метод стрельбы эффективнее в любом случае, потому что все эти задачи Коши решаются независимо, то есть каждый шаг сетки выполняется быстрее.

85. **Вопрос:** Есть ли смысл при реализации метода параллельной стрельбы использовать распараллеливание решения системы линейных уравнений в процессе итераций по пристрелочным параметрам?

Ответ: Про метод стрельбы можно почитать в предыдущем пункте. По сути он представляет из себя огромную систему ОДУ (если исходная система состояла из m уравнений, то новая будет состоять из $m * n$, где n – количество итераций пристрелки). Если есть способ распараллелить её эффективнее, чем в n раз, то почему бы и нет.

86. **Вопрос:** При реализации параллельной пристрелки реализовано две версии метода. При первой реализации сначала решаются нелинейные уравнения, данные сохраняются, а затем решаются уравнения в вариациях, во второй реализации все эти уравнения решаются одновременно. Какая реализация более эффективна с точки зрения распараллеливания?

Ответ: Во втором случае - отрезок параметра разбивается между исполнителями и в каждом из отрезков можно задать задачу Коши, и решение такой задачи может осуществляться независимо на каждом исполнителе.

r.s. Непонятно, какие данные сохраняются. Просто как факт, никакая синхронизация между процессами не требуется, после разбиения отрезка параметра между исполнителями.

87. **Вопрос:** Решаются две близкие задачи – уравнение диффузии и уравнение диффузии с конвекцией. Используется полностью неявная схема. Можно ли в каждом из этих случаев использовать алгоритм редукции для решения системы линейных уравнений с трехдиагональной матрицей?

Ответ: Да, можно. Редукция - это эквивалентное математическое преобразование и на точность решения системы линейных уравнений не влияет. Погрешность решения появляется, только в момент лианеризации схемы. Неявная схема обычно устойчива (почти при всех параметрах сетки), поэтому лианеризация схемы и последующее ее

решение как системы линейных уравнений даст решение в пределах ошибки аппроксимации.

p.s.(это возможно неверно)Также можно заметить, что количество арифметических операций(сложение, умножение и т.п) для вычисления значения функции в заданной точке одинаково в однопоточной версии прогонки так и после редукции(это следует из обратимости математического преобразования).

88. **Вопрос:** Решаются две близкие задачи – уравнение диффузии и уравнение диффузии с конвекцией. Для задачи с конвекцией используются схемы, в которых диффузионный оператор аппроксимируется на верхнем слое по времени, а конвективные члены – на нижнем слое по времени. Для уравнения диффузии использована неявная схема. Можно ли в каждом из этих случаев использовать алгоритм редукции для решения системы линейных уравнений с трехдиагональной матрицей?

Ответ: Да, можно. (см.87) Во втором случае(с конвекцией) погрешность решения может оказаться больше из-за ошибки вычисления конвекционной части на предыдущем слое. В данном случае схема тоже неявная, поэтому скорее всего устойчива.(см. пункт 87)

89. **Вопрос:** Линейное уравнение переноса решается с помощью простой явной разностной схемы на трехточечном шаблоне. Используется геометрическое распараллеливание. Сравнить варианты с перекрытием и без перекрытия зон ответственности с точки зрения эффективности распараллеливания.

Ответ: нет ответа.

90. **Вопрос:** Линейное уравнение переноса решается с помощью явной разностной схемы повышенного порядка аппроксимации на двухслойном шаблоне. Используется геометрическое распараллеливание. Сравнить варианты с перекрытием и без перекрытия зон ответственности исполнителей с точки зрения эффективности распараллеливания.

Ответ: нет ответа.

91. **Вопрос:** Линейное уравнение переноса решается с помощью явной разностной схемы повышенного порядка аппроксимации на трехслойном шаблоне. Используется геометрическое распараллеливание. Сравнить варианты с перекрытием и без перекрытия зон ответственности исполнителей с точки зрения эффективности распараллеливания.

Ответ: Здесь необходим обмен данными в обоих направлениях, так как последнему исполнителю не хватит данных на правой границе, что приведет к использованию там другой разностной схемы, что в свою очередь может привести к разбалансировке загрузки исполнителей. В случае перекрытия нужно лишь незначительно увеличить зоны перекрытия, что представляется наиболее предпочтительным вариантом с точки зрения эффективности.

Комментарий:

Ain't got nothing better.

92. **Вопрос:** При использовании бикомпактной разностной схемы для решения уравнения Хопфа объем передачи необходимой информации по сравнению с реализацией явной двухслойной разностной схемы того же порядка при каждом обмене: А. Уменьшается в два раза Б. Увеличивается в два раза В. Остается неизменным.

Ответ: А

Комментарий:

Ничего конкретного нигде не нашел, но логика подсказывает, что раз схема бикомпактная, то что-то становится более компактным в 2 раза. :)

(С другой стороны википедия говорит, что компактные схемы просто имеют больший порядок аппроксимации. Но здесь порядки аппроксимации совпадают, а устроена компактная схема может быть куда сложнее, поэтому и пересылок может быть больше.)

93. **Вопрос:** Разностная схема бегущего счета А. Не может быть эффективно распараллелена Б. Может быть эффективно распараллелена с помощью геометрического распараллеливания В. Может быть эффективно распараллелена с помощью геометрического распараллеливания, но требуется значительная модификация в зоне перекрытия областей ответственности исполнителей Г. Может быть распараллелена с использованием конвейера Д. Надо использовать редукционный метод для решения уравнений на верхнем слое Е Г. Может быть распараллелена с использованием комбинации идей конвейера и геометрического параллелизма.

Ответ: Г

Комментарий:

Судя по тому, что написано здесь: http://neo-chaos.narod.ru/useful/nummethod/kalitkin_10_334-367.pdf, такую схему можно считать последовательно слева направо, поскольку для очередного значения на текущем слое, нужно значение текущего слоя, которое левее, и несколько значений с предыдущего слоя. Геометрическое распараллеливание отпадает, конвейерное подходит. Что имеется в виду в Д не особо ясно.

Впрочем в бумажке был написан ответ Е.

94. **Вопрос:** При использовании явных компактных разностных схем при решении задач для систем уравнений гиперболического типа на параллельной системе для повышения производительности целесообразно использовать А. Геометрическое распараллеливание с перекрытием зон ответственности исполнителей Б. Редукционный метод решения систем линейных алгебраических уравнений В. Итерационное обращение оператора вместо точного Г. Обращение оператора по отдельным блокам

Ответ: А

Комментарий:

В бумажке был ответ Б, но имхо, если схема явная, то геометрическое распараллеливание подходит хорошо.

95. **Вопрос:** При использовании явного разрывного метода Галеркина при решении задач для систем уравнений гиперболического типа на параллельной системе для повышения производительности целесообразно использовать А. Геометрическое распараллеливание с перекрытием зон ответственности исполнителей Б. Редукционный метод решения систем линейных алгебраических уравнений В. Итерационное обращение оператора вместо точного Г. Обращение оператора по отдельным блокам

Ответ: А

Комментарий:

Согласно цитате
([ftp://ftp.gentoo.ru/local/57/6_term/%D0%A2%D0%B5%D0%BE%D1%80%D0%B8%D1%8F%20%D0%B0%D0%BB%D0%B3%D0%BE%D1%80%D0%B8%D1%82%D0%BC%D0%BE%D0%B2/Group B/GPU_8.pptx](ftp://ftp.gentoo.ru/local/57/6_term/%D0%A2%D0%B5%D0%BE%D1%80%D0%B8%D1%8F%20%D0%B0%D0%BB%D0%B3%D0%BE%D1%80%D0%B8%D1%82%D0%BC%D0%BE%D0%B2/Group%20B/GPU_8.pptx)): “Разрывный метод Галёркина легко распараллелить за счет разделения области на конечные элементы, вычисления на которых можно производить независимо.” Видимо здесь имеется в виду геометрическое распараллеливание.

96. **Вопрос:** Задача решалась численно в трехмерной области в форме параллелепипеда. При решении уравнения параболического типа с помощью явной разностной схемы было найдено оптимальное разбиение на зоны ответственности при использовании

геометрического распараллеливания. Будет ли это разбиение также оптимальным при использовании явных разностных схем на том же шаблоне для решения задачи гиперболического типа?

Ответ: Да, будет. Решения обоих этих уравнений (параболическое и гиперболическое) зависят от времени, этим они похожи. Отличие в характеристиках, но это проявляется лишь на уровне координат, на обмен данными между процессорами это не влияет, отсюда свойство оптимальности должно сохраниться.

97. **Вопрос:** При использовании явной компактной разностной схемы третьего порядка аппроксимации для решения уравнения переноса объем передачи необходимой информации по сравнению с реализацией явной двухслойной разностной схемы того же порядка при каждом обмене: А. Уменьшается в два раза Б. Увеличивается в два раза В. Остается неизменным.

Ответ: Уменьшается в 2 раза.

Чтобы второй схемой сделать тот же шаг по времени, что и первой схемой, нужно рассчитать два слоя, в то время как первая схема считает лишь один слой. Если считать, что объем передаваемой информации от слоя к слою приблизительно одинаков, то ответ становится прозрачным.

98. **Вопрос:** Линейное уравнение переноса решается с помощью явной компактной разностной схемы третьего порядка аппроксимации на двухслойном шаблоне. Используется геометрическое распараллеливание. Сравнить варианты геометрического распараллеливания с перекрытием и без перекрытия зон ответственности исполнителей с точки зрения эффективности распараллеливания.

Ответ: Обмен между зонами ответственности исполнителей не требуется, т.е. перекрытие нецелесообразно, а значит менее эффективно. В случае линейного уравнения переноса характеристиками являются прямые линии. Соответственно, если перейти к переменным Римана («встать на характеристику»), то значение на ней пересчитывать не надо, оно сохраняется на всей характеристике. Значит, обмен между зонами ответственности исполнителей не нужен.

99. **Вопрос:** При использовании гибридной разностной схемы с геометрическим распараллеливанием для сокращения числа обменов данными между процессорами используется схема первого порядка аппроксимации. Описать достоинства и недостатки такого подхода.

Ответ: Достоинства: Простота кода (схема первого (!) порядка)

Недостатки: Низкий порядок аппроксимации схемы (первый, можно выше)

100. **Вопрос:** При использовании бикompактной разностной схемы с геометрическим распараллеливанием для повышения эффективности распараллеливания используется явная схема первого порядка аппроксимации. Описать достоинства и недостатки такого подхода.

Ответ: Достоинства: Простота кода (явная (!) схема первого (!) порядка)

Недостатки: Низкий порядок аппроксимации (первый, хотя на бикompактных схемах можно и пятый получить).